



Capstone Project 1

CMU-SE 450

Project Proposal

Version 1.2

Date: August 10, 2026

COVAI

TEST COVERAGE ANALYSIS AND AI-DRIVEN TEST GENERATION SYSTEM

Created by C1SE.30

Dinh, Huynh Tan

Phuc, Tran Huu

Quy, Nguyen Duy

Thuan, Ngo Huu

Approved by

Man, Nguyen Duc, Ph.D.

Proposal Review Panel Representative:

Name	Signature	Date
------	-----------	------

Capstone Project 1- Mentor:

Name	Signature	Date
------	-----------	------

PROJECT INFORMATION

Project acronym	COVAI		
Project Title	Test Coverage Analysis and AI-Driven Test Generation System		
Start Date	Aug 10, 2026	End Date	Dec 12, 2026
Lead Institution	International School, Duy Tan University		
Project Mentor	Man, Nguyen Duc, Ph.D.		
Scrum master / Project Leader & contact details	Dinh, Huynh Tan Email: huynhtandinh.dev@gmail.com Tel: 0365 472 162 ID: 29219052500		
Partner Organization	Duy Tan University		
Project Web URL			
Team members	Name	Email	Tel
29219052500	Dinh, Huynh Tan	huynhtandinh.dev@gmail.com	0365472162
29211458179	Phuc, Tran Huu	phuc12435az@gmail.com	0842644169
29211158823	Quy, Nguyen Duy	quylavip333@gmail.com	0334814522
29219065178	Thuan, Huu Ngo	thuanhuugl@gmail.com	0385591447

DOCUMENT APPROVALS

The following signatures are required for approval of this document.

Dinh, Huynh Tan Student ID: 29219052500 <i>Scrum Master</i>	Signature	Date
Phuc, Tran Huu Student ID: 29211458179 <i>Team Member</i>	Signature	Date
Quy, Nguyen Duy Student ID: 29211158823 <i>Team Member</i>	Signature	Date
Thuan, Ngo Huu Student ID: 29219065178 <i>Team Member</i>	Signature	Date

PROJECT PROPOSAL DOCUMENT

Document Title	Project Proposal Document		
Author(s)	Dinh, Huynh Tan		
Role	Scrum Master		
Date	August 10, 2026	File name:	C1SE.30_Proposal_ver1.2.doc

REVISION HISTORY

Version	Date	Comments	Author
1.0	Aug 10, 2026	Initial Proposal Document	Dinh
1.1	Sep 05, 2026	Update Document	C1SE.30
1.2	Sep 07, 2026	Update Header and Footer	C1SE.30

TABLE OF CONTENTS

1. Project Title.....	1
2. Project Overview	1
3. Project Background and Motivation	1
4. Problem Statement	2
5. Proposed Solution	3
6. Related Works or Projects on the Market	4
7. System Architecture Overview.....	5
7.1 System Context Diagram.....	5
7.2 System Workflow	9
7.3 Database Design Overview.....	11
8. Objectives and Deliverables.....	12
8.1 General Objective	12
8.2 Specific Objectives	12
8.3 In Scope	13
8.4 Out of Scope	14
9. Features & Requirements.....	14
9.1 Software Testing Features	14
9.2 AI Chatbot Trustworthiness Evaluation Features.....	14
9.3 Functional Requirements	15
9.4 Non-Functional Requirements.....	16
9.5 AI Trustworthiness Evaluation Framework	17
9.6 Evaluation Dataset, Metrics and Scoring	18
10. Constraints and Assumptions.....	19
11. Target Users and Stakeholders	20
12. Development Methodology and Technology Stack	20
12.1 Development Methodology (Agile Scrum)	20
12.2 Technology Stack	21
13. Timeline	22
14. Project Team	23
15. Risk Management.....	24
16. Budget and Resources	27

16.1	Budget.....	27
16.2	Human Resources	28
17.	Conclusion	28
18.	References	30

LIST OF FIGURES

Figure 1- System context diagram.....	5
Figure 2 - System Workflow	9
Figure 3 - System Workflow	10
Figure 4 - Database Design Overview.....	11
Figure 5 - Database Design Overview.....	12

LIST OF TABLES

Table 1 - Solution Comparison.....4

Table 2 - Specific Objectives 12

Table 3 - Functional Requirements 15

Table 4 - Non-Functional Requirements 16

Table 5 - Three-Layer AI Chatbot Evaluation Framework..... 18

Table 6 - Evaluation Dataset, Metrics and Scoring..... 18

Table 7 - Constraints and Assumptions..... 19

Table 8 - Target Users and Stakeholders.....20

Table 9 - Technology Stack.....21

Table 10 - Project Schedule.....22

Table 11 - Project Team Roles and Responsibilities.....23

Table 12 - Risk Assessment24

Table 13 - Budget and Resource Allocation27

Table 14 - References30

1. Project Title

COVAI – Software Testing and AI Chatbot Trustworthiness Evaluation System

2. Project Overview

COVAI is an AI-driven software quality platform that combines two complementary capabilities in a single engineering workflow. The first capability analyzes automated software testing by ingesting source code, detecting supported testing frameworks, executing tests in isolated environments, analyzing coverage and code structure, and generating additional tests. The second capability evaluates the trustworthiness of an internal AI chatbot according to project-specific knowledge, business rules, API contracts, conversation context and tool permissions.

The platform is intended for developers, QA engineers and AI engineers who need repeatable evidence about software quality. COVAI supports local ZIP uploads and GitHub repositories, project snapshots, background jobs, dashboards and notifications. The platform initially focuses on JavaScript/TypeScript projects and supported testing frameworks such as Jest, Vitest, Supertest, Playwright and Cypress.

3. Project Background and Motivation

Software testing is essential for ensuring software quality, but as projects become larger and more complex, developers face increasing difficulties in writing, executing, and maintaining automated tests. They often need to manually analyze test coverage, identify insufficiently tested code, understand code complexity, and create additional test cases.

Modern projects may also use different testing levels and frameworks, such as Unit, Integration, and System Testing with Jest, Vitest, Supertest, Playwright, or Cypress. This makes automated framework detection, test execution, and result analysis important challenges.

At the same time, many applications integrate AI chatbots for knowledge assistance, user guidance, and tool-based actions. These chatbots must be evaluated not only for answer correctness, but also for relevance, context retention, safety, privacy, and compliance with project-specific rules.

Motivation:

- Reduce manual effort in software testing and coverage analysis.

- Identify insufficiently tested and complex source-code areas.
- Use AI to assist in generating additional test cases.
- Support multiple testing levels and frameworks in one platform.
- Evaluate internal AI chatbots using project-specific knowledge and rules.
- Provide measurable results and recommendations for improving software and chatbot quality.
- Automate analysis through GitHub integration.

4. Problem Statement

Modern software teams face two related quality problems. First, automated testing is distributed across different testing levels and frameworks. Developers must manually execute tests, inspect coverage reports, identify insufficiently tested code, understand complex control flow, and create additional tests. These activities become increasingly time-consuming as projects grow and change frequently.

Second, web applications are increasingly integrating internal AI chatbots for question answering, user guidance, and tool or API-based actions. The reliability of these chatbots cannot be determined only by the name of the underlying model. Their behavior also depends on system prompts, internal data, retrieval mechanisms, conversation context, tool permissions, refusal logic, and implementation choices. Two chatbots that use the same model may therefore produce different levels of reliability.

A general-purpose comparison of large AI models is too broad for this project because different tasks require different evaluation criteria and many open-ended prompts do not have a single ground-truth answer. In contrast, an internal chatbot operates inside a specific application where source code, business documents, knowledge bases, API specifications, and confirmed business rules can provide project-specific ground truth.

Therefore, COVAI addresses the need for a unified engineering platform that can analyze software testing quality and also evaluate whether an internal AI chatbot behaves correctly, faithfully, safely, consistently, and appropriately within the data and business rules of its own web application.

5. Proposed Solution

COVAI is an AI-driven software quality platform that combines automated software testing and code analysis with AI chatbot trustworthiness evaluation in a unified engineering workflow.

The system accepts source code through local ZIP upload or GitHub repository import. After a project is imported, COVAI creates a source-code snapshot and analyzes the project configuration to identify the supported testing framework.

For software testing, COVAI executes tests inside isolated Docker environments and collects test results, execution logs, and coverage reports. Coverage is analyzed at overall, file, and function levels. The system also analyzes source-code structure using Control Flow Graph (CFG) and Cyclomatic Complexity to identify complex and insufficiently tested areas.

When testing gaps are identified, the AI service analyzes the source code, test results, coverage information, and code structure to generate additional test skeletons or test implementations. Generated tests can then be reviewed and validated by the user.

For AI chatbot evaluation, COVAI uses project-specific knowledge, business rules, API specifications, and expected behaviors to build an evaluation profile and generate benchmark scenarios. The system evaluates internal chatbots across tasks such as Knowledge Q&A, Usage Guidance, and Action/Tool Execution. Evaluation results include criteria such as correctness, faithfulness, relevance, consistency, context retention, safety, privacy, appropriate refusal, and tool usage.

Core Components:

- Project Management & Source Ingestion: Manages projects, source-code snapshots, ZIP uploads, and GitHub repository imports.
- Test Execution & Coverage Analysis: Detects supported testing frameworks, executes tests in isolated Docker environments, and collects and analyzes coverage reports.
- Code Structure Analysis: Analyzes source code using Control Flow Graph and Cyclomatic Complexity.
- AI Test Generation: Uses AI to generate test skeletons and full test implementations based on source code and analysis results.

- Multi-Framework Testing: Supports selected testing levels and frameworks, including Jest, Vitest, Supertest, Playwright, and Cypress.
- AI Chatbot Trustworthiness Evaluation: Builds project-specific evaluation profiles, generates benchmark scenarios, evaluates chatbot responses and actions, and calculates trustworthiness scores.
- Evidence, Dashboard & Reporting: Stores test and evaluation evidence and provides visual reports, scores, failed cases, risk indicators, recommendations, and job status.
- GitHub Automation: Integrates with GitHub to import repositories and trigger analysis when source code is updated.

6. Related Works or Projects on the Market

Table 1 - Solution Comparison

Solution	Description	Strengths	Weaknesses
SonarQube	Platform for source-code quality and static analysis	Mature code-quality analysis	Limited focus on automated test generation
Codecov	Platform for collecting and visualizing test coverage	Strong coverage reporting	Does not provide complete test-generation workflow
Jest	JavaScript testing framework	Test execution and coverage support	Focused on test execution rather than complete project analysis
Playwright	Browser automation and testing framework	Powerful end-to-end testing	Not a complete coverage and AI analysis platform

Solution	Description	Strengths	Weaknesses
CovAI	AI-driven platform for test execution, coverage analysis, code analysis, and test generation	Integrate testing, coverage, code analysis, and AI assistance	Initial implementation focuses mainly on JavaScript-based projects

7. System Architecture Overview

7.1 System Context Diagram

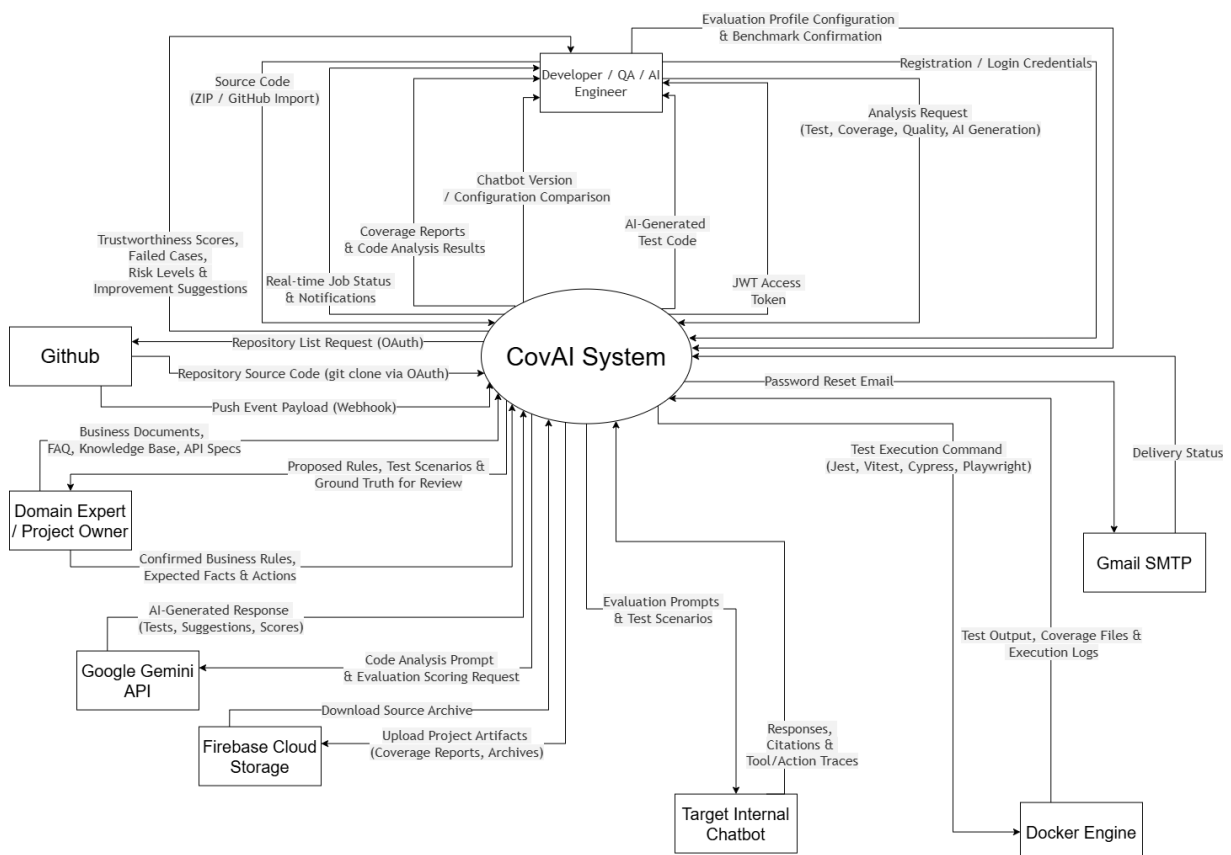


Figure 1- System context diagram

System Context Description:

Developer / QA / AI Engineer

The Developer / QA / AI Engineer is the primary user of COVAI who uses the platform to analyze software quality and evaluate internal AI chatbot trustworthiness:

- Login / Logout

- Create and manage projects
- Upload source code (ZIP / RAR)
- Import GitHub repositories
- Select or detect testing framework
- Trigger test analysis
- View coverage results (statement, branch, function, line)
- View CFG and Cyclomatic Complexity
- Request AI-generated tests (skeletons and full implementations)
- View generated test history
- Configure evaluation profile and chatbot task types
- Confirm or edit benchmark test scenarios
- View trustworthiness scores (overall and component)
- View failed cases, error classification, and risk levels
- View improvement suggestions
- Compare chatbot versions or configurations
- Receive real-time job status and notifications

Domain Expert / Project Owner

The Domain Expert / Project Owner provides and confirms the project-specific ground truth used to evaluate internal AI chatbots:

- Upload business documents, FAQ, knowledge base, and API specifications
- Confirm or edit business rules, expected facts, and expected actions
- Review proposed evaluation rules and test scenarios generated by the system
- Confirm risk levels and forbidden actions for high-risk scenarios

GitHub

GitHub is an external service used to provide source-code repositories and repository update events:

- Provide repository listing (via OAuth)
- Provide repository source code (via git clone)
- Send push event payloads when repositories are updated (Webhook)

Docker Environment

The Docker environment provides an isolated execution environment for running user projects and tests safely:

- Install project dependencies
- Execute testing frameworks (Jest, Vitest, Supertest, Playwright, Cypress)
- Generate coverage reports and test output
- Capture execution logs (stdout, stderr)

AI Service (Google Gemini API)

The AI service provides intelligent source-code analysis, test-generation, and evaluation-scoring capabilities:

- Analyze source code and coverage data
- Generate test skeletons
- Generate full test implementations
- Provide suggestions for insufficiently tested code
- Score chatbot responses against evaluation criteria and ground truth

Target Internal Chatbot

The Target Internal Chatbot is the external chatbot system being evaluated. It is integrated into a specific web application and operates within that application's data and business rules:

- Receive evaluation prompts and test scenarios from COVAI
- Return responses, citations, and available tool/action execution traces
- Provide latency and metadata when available

Firebase Cloud Storage

Firebase Cloud Storage is an external cloud storage service used to persist project artifacts:

- Store uploaded source archives (ZIP / RAR)
- Store coverage report artifacts (JSON, LCOV)
- Provide download access to stored artifacts

Gmail SMTP

Gmail SMTP is an external email delivery service used for account management:

- Deliver password reset emails to users
- Return delivery status

COVAI System

COVAI processes project source code, executes tests, analyzes results, provides quality insights to developers, and evaluates the trustworthiness of internal AI chatbots against project-specific data and business rules:

- Project and snapshot management
- Test framework detection (Jest, Vitest, Supertest, Playwright, Cypress)
- Test execution in isolated Docker environments
- Coverage analysis (statement, branch, function, line at overall / file / function level)
- Control Flow Graph analysis
- Cyclomatic Complexity analysis
- AI-assisted test generation (skeletons and full implementations)
- AI evaluation knowledge ingestion (source code, documents, knowledge base, API specs)
- Evaluation profile building and task classification
- Benchmark and test scenario generation
- Chatbot evaluation execution (prompt delivery, response collection)
- Multi-criteria trustworthiness scoring (correctness, faithfulness, hallucination, safety, etc.)
- Diagnosis and improvement suggestions
- Chatbot version and configuration comparison
- Job management and background processing
- Real-time notifications
- GitHub integration and webhook automation
- User authentication and password reset

7.2 System Workflow

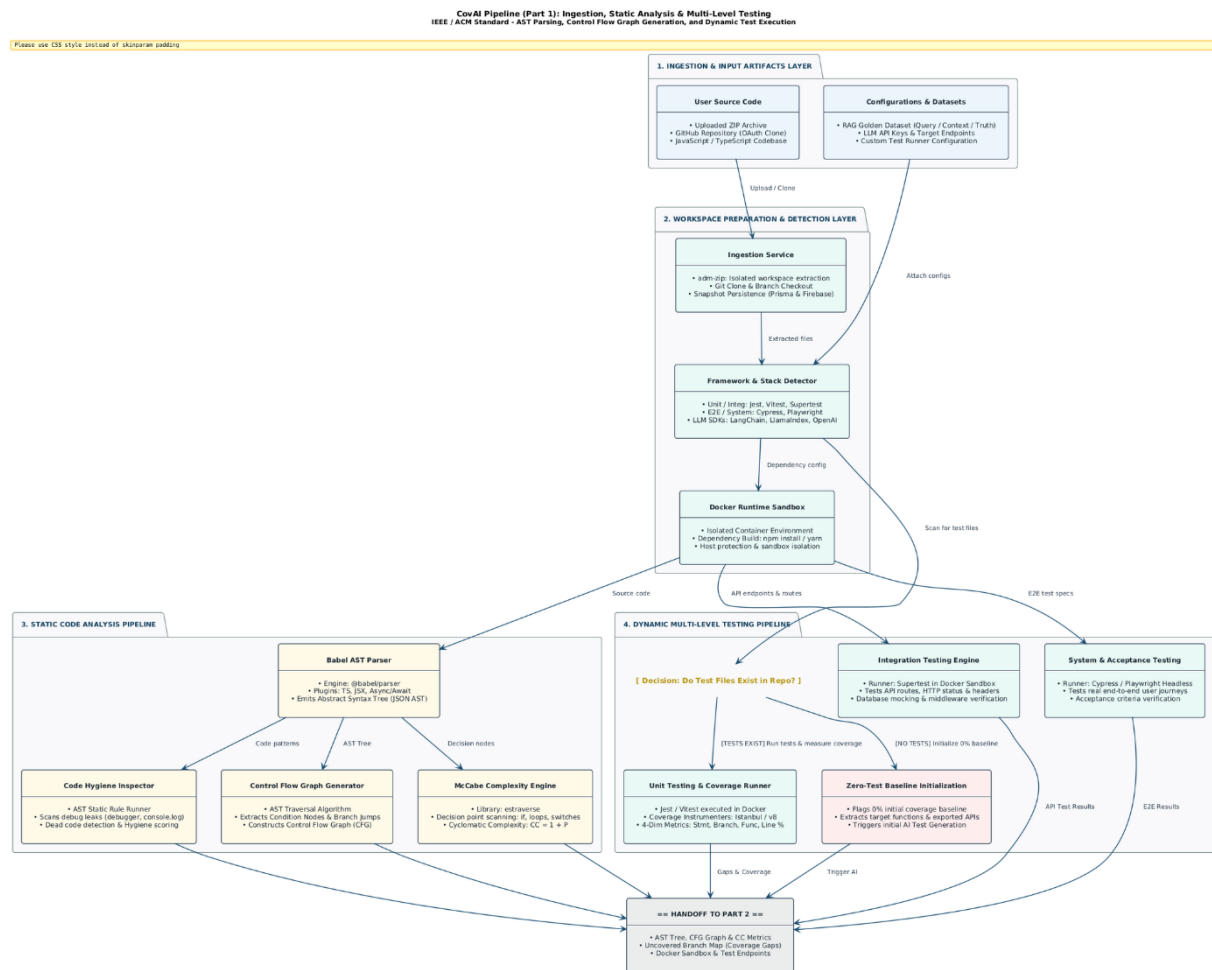


Figure 2 - System Workflow

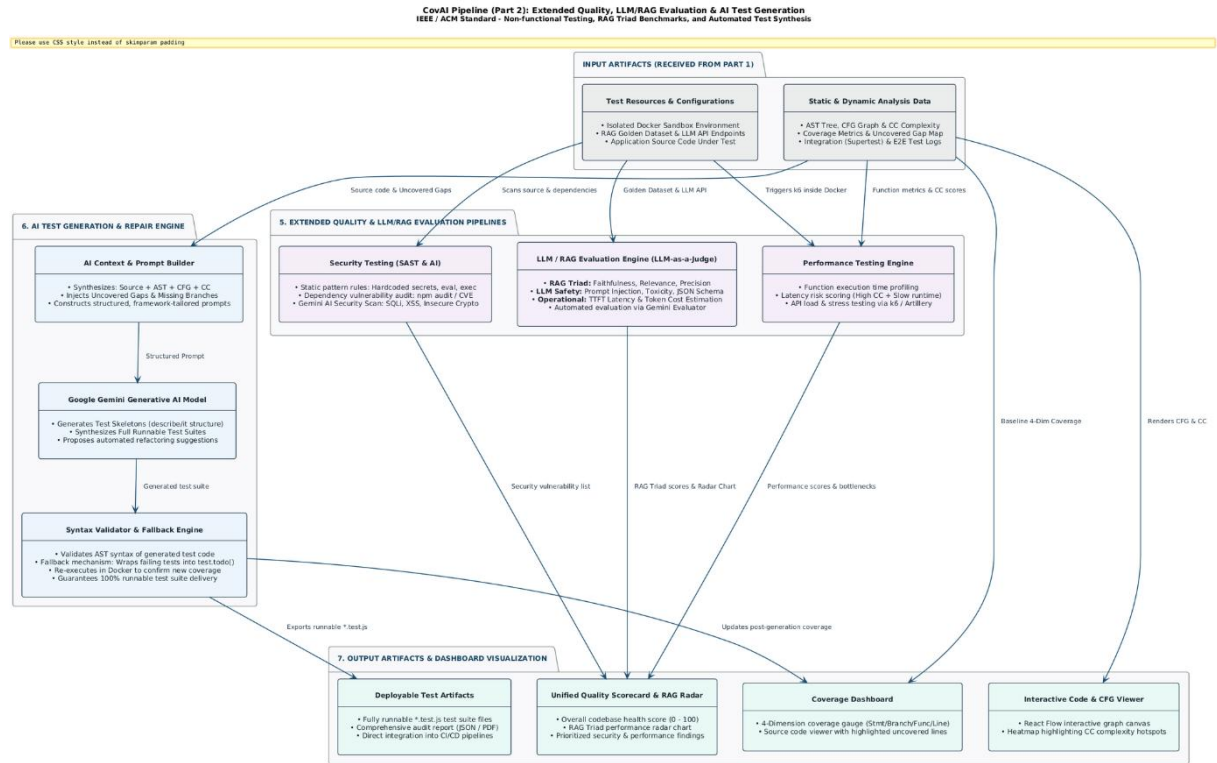


Figure 3 - System Workflow

7.3 Database Design Overview

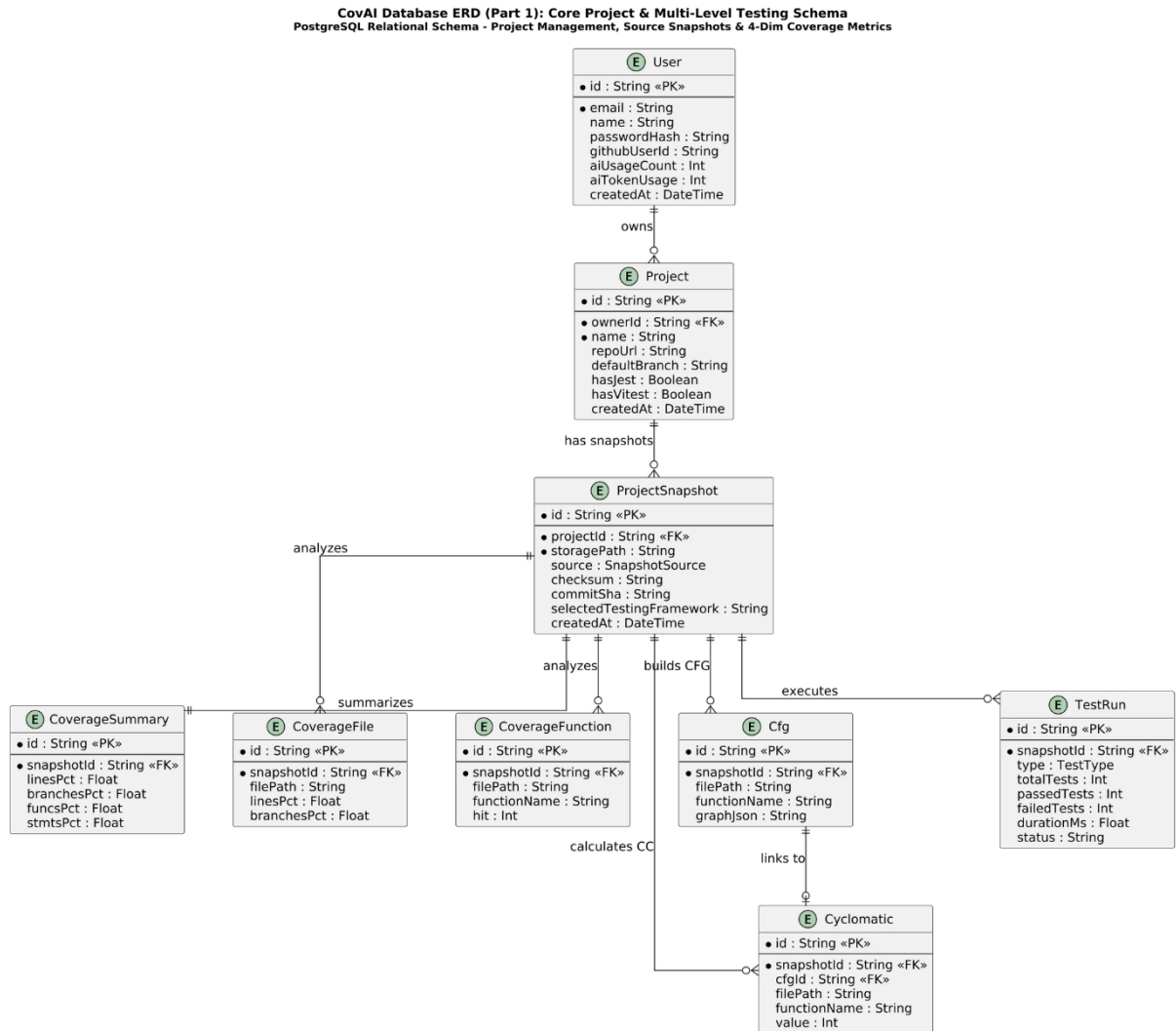


Figure 4 - Database Design Overview

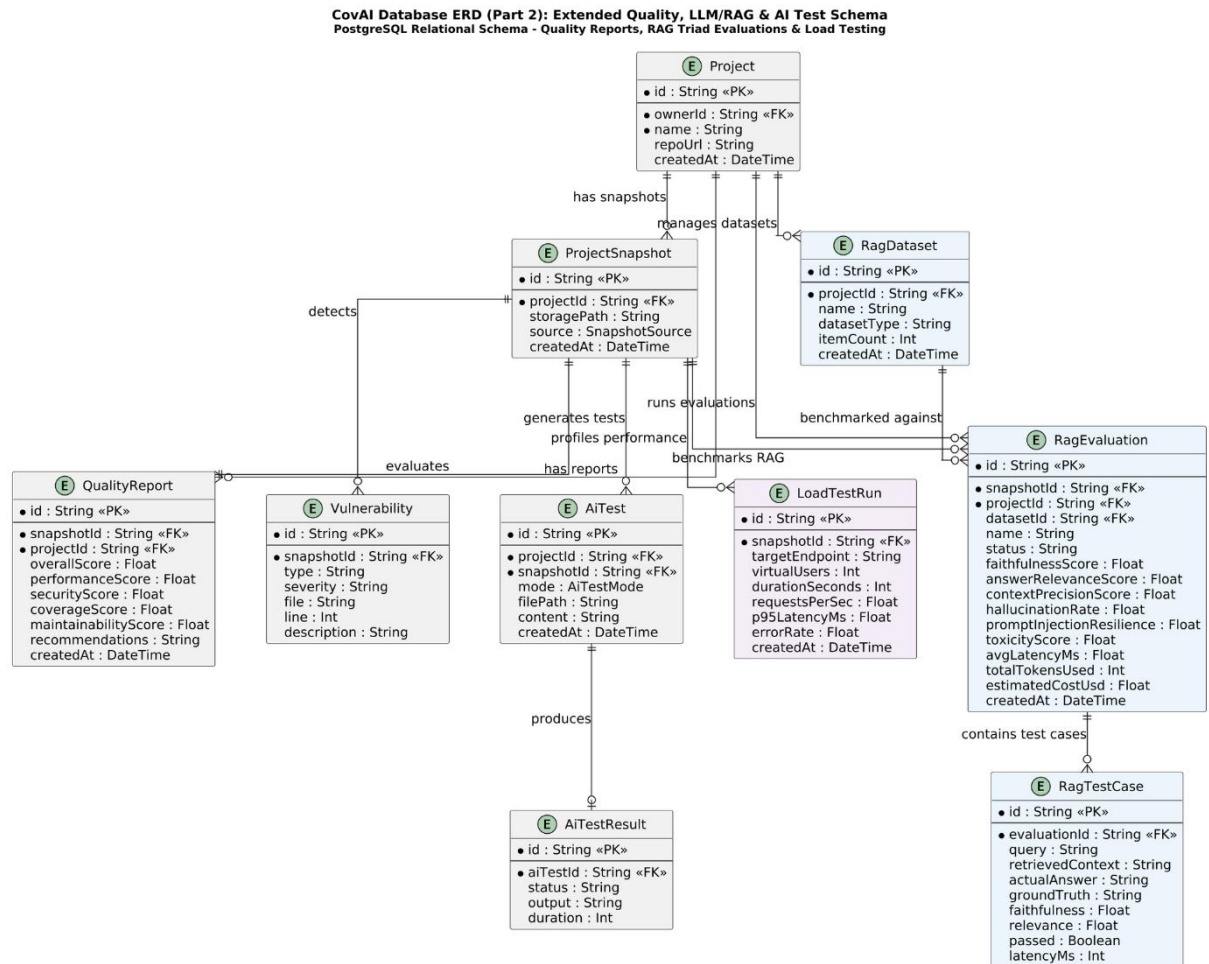


Figure 5 - Database Design Overview

8. Objectives and Deliverables

8.1 General Objective

To develop an AI-driven software quality platform that automates software test analysis and provides evidence-based evaluation of internal AI chatbot trustworthiness using project-specific knowledge, rules and execution context.

8.2 Specific Objectives

Table 2 - Specific Objectives

ID	Objective	Description
SO-01	Project and Source Code Management	Implement project, source-code, repository and snapshot management.
SO-02	Automated Test Execution	Execute supported test frameworks inside controlled Docker environments.

SO-03	Test Coverage Analysis	Analyze statement, branch, function and line coverage at supported scopes.
SO-04	Code Structure Analysis	Generate Control Flow Graphs and calculate Cyclomatic Complexity.
SO-05	AI-Assisted Test Generation	Generate test skeletons and runnable test implementations from analysis evidence.
SO-06	Multi-Framework Testing	Support the selected Unit, Integration and System testing frameworks.
SO-07	GitHub Automation	Trigger analysis from repository updates through GitHub integration and webhooks.
SO-08	AI Chatbot Evaluation Framework	Provide reusable mechanisms for evaluating internal chatbot behavior.
SO-09	Project-Specific Evaluation Profile	Build and confirm evaluation rules from project knowledge and business context.
SO-10	Benchmark and Scenario Generation	Generate knowledge, guidance and action/tool evaluation scenarios.
SO-11	Trustworthiness Scoring	Compute criterion-level, task-level and overall evaluation scores.
SO-12	Diagnosis and Improvement Recommendations	Classify likely failure causes and suggest improvement directions.
SO-13	Comparison and Reporting	Compare chatbot versions/configurations under the same evaluation profile and report results.

8.3 In Scope

- JavaScript/TypeScript project ingestion and snapshot management.
- Jest, Vitest, Supertest, Playwright and Cypress integrations within the agreed project scope.
- Docker-based test execution, logs and resource controls.
- Coverage, CFG and Cyclomatic Complexity analysis.

- AI-assisted test generation with validation/fallback behavior.
- Project-specific AI chatbot evaluation for Knowledge Q&A, Usage Guidance and Action/Tool Execution.
- Trustworthiness scoring, diagnosis, comparison and reporting.
- GitHub repository import and webhook-triggered analysis.

8.4 Out of Scope

- Full support for every programming language or test framework.
- Universal benchmarking of all public foundation models.
- Uncontrolled execution of real-world production actions during chatbot evaluation.
- Guaranteeing that AI-generated tests or AI judgments are always correct.
- Replacing human/domain-expert validation for critical business rules and high-risk scenarios.

9. Features & Requirements

9.1 Software Testing Features

- Project & Source Management
- Framework Detection
- Isolated Test Execution
- Coverage Analysis
- Control Flow Graph Analysis
- Cyclomatic Complexity Analysis
- AI-Assisted Test Generation
- Multi-Framework Testing
- GitHub Automation
- Dashboard, Jobs and Notifications

9.2 AI Chatbot Trustworthiness Evaluation Features

- AI Evaluation Knowledge Ingestion
- Evaluation Profile Builder
- Benchmark & Scenario Generation
- Chatbot Evaluation Runner
- Evidence Collection

- Trustworthiness Scoring
- Diagnosis & Improvement Suggestions
- Comparison & Reporting

9.3 Functional Requirements

Table 3 - Functional Requirements

ID	Functional Requirement
FR-01	The system shall authenticate users and allow authorized users to create and manage projects.
FR-02	The system shall accept project source code through local ZIP upload or GitHub repository import and maintain project snapshots.
FR-03	The system shall detect supported testing frameworks and execute tests inside isolated Docker environments.
FR-04	The system shall calculate statement, branch, function, and line coverage and present results at overall, file, and function levels when supported by the framework.
FR-05	The system shall analyze source code using Control Flow Graph and Cyclomatic Complexity techniques.
FR-06	The system shall allow users to request AI-generated test skeletons and full test implementations and store generation history.
FR-07	The system shall integrate with GitHub and support automated analysis triggered by repository updates.
FR-08	The system shall allow users to provide business documents, knowledge-base content, API specifications, and project rules for chatbot evaluation.
FR-09	The system shall analyze the provided project context and propose an evaluation profile containing chatbot task types, relevant rules, and candidate test scenarios.
FR-10	The system shall allow a user or domain expert to confirm or edit important rules, expected facts, and expected actions before final evaluation.

ID	Functional Requirement
FR-11	The system shall support evaluation of knowledge Q&A, usage guidance, and action/tool-execution tasks.
FR-12	The system shall send evaluation prompts to a configurable target chatbot interface and collect returned answers and available execution metadata.
FR-13	The system shall evaluate core criteria including correctness, faithfulness, relevance, completeness, hallucination, consistency, context retention, clarification, refusal, safety, privacy, latency, and cost where measurable.
FR-14	For action/tool tasks, the system shall check tool selection, parameter accuracy, required confirmation, forbidden actions, and duplicate execution where relevant.
FR-15	The system shall display overall and component scores, failed cases, serious violations, risk levels, error categories, and improvement suggestions.
FR-16	The system shall support comparison of multiple chatbot versions or configurations under the same evaluation profile.

9.4 Non-Functional Requirements

Table 4 - Non-Functional Requirements

ID	Quality Attribute	Non-Functional Requirement
NFR-01	Security	Source code, business documents, credentials, and chatbot evaluation data shall be accessible only to authorized users and shall not be exposed unnecessarily to external services.
NFR-02	Isolation	User projects and test executions shall run inside controlled Docker environments with timeout and resource limits.
NFR-03	Reproducibility	Evaluation runs shall store relevant project snapshot, evaluation profile, test cases, and available chatbot

ID	Quality Attribute	Non-Functional Requirement
		configuration metadata so results can be reviewed and compared.
NFR-04	Reliability	Failures in one background job or target chatbot request shall be handled without corrupting other project results; failed jobs shall provide logs or status information.
NFR-05	Performance	Long-running analysis shall use background jobs and shall provide progress/status feedback rather than blocking the user interface.
NFR-06	Modularity	The chatbot evaluation framework shall support adding new task-specific evaluation modules in future versions without redesigning the entire platform.
NFR-07	Usability	Evaluation results shall explain component scores, failed cases, expected behavior, observed behavior, and recommended next steps in a form understandable to developers and evaluators.
NFR-08	Safety	Evaluation of tool/action chatbots shall avoid unintended real-world side effects by using confirmation rules and controlled test environments when necessary.

9.5 AI Trustworthiness Evaluation Framework

The AI evaluation extension uses a three-layer framework so that COVAI can remain reusable across different web applications while still respecting project-specific business rules.

Table 5 - Three-Layer AI Chatbot Evaluation Framework

Layer	Purpose	Examples
Layer 1 – Core Criteria	Applied to all evaluated chatbots.	Correctness; Faithfulness; Relevance; Completeness; Hallucination; Consistency; Context Retention; Clarification; Appropriate Refusal; Safety; Privacy; Latency; Cost.
Layer 2 – Task Module	Adds criteria based on chatbot behavior type.	Knowledge Q&A: source correctness and no fabricated facts/policies. Usage Guidance: correct UI/menu/button/step sequence and ability to help complete the task. Action/Tool: correct tool/API, correct parameters, required confirmation, and no duplicate or forbidden action.
Layer 3 – Project Rules	Uses the specific application's ground truth.	Business rules, policies, API contracts, role/permission rules, knowledge-base facts, and expected actions confirmed for the target project.

9.6 Evaluation Dataset, Metrics and Scoring

Each evaluation case is defined by a task type, prompt, context, expected behavior and evidence requirements. For deterministic cases, the expected answer, facts, UI steps, tool calls or forbidden actions can be specified explicitly. User/domain-expert confirmation is used for important project rules.

Table 6 - Evaluation Dataset, Metrics and Scoring

Area	Examples
Dataset	Knowledge Q&A cases, Usage Guidance cases, Action/Tool cases, refusal/safety cases, context-retention cases.
Ground Truth	Expected facts, expected answer elements, expected UI sequence, allowed tools, expected parameters, forbidden actions.

Area	Examples
Evidence	Chatbot answer, citations, conversation context, tool/API trace, parameters, confirmation events, latency and available cost metadata.
Scoring	Criterion score → task score → aggregate score. Deterministic checks are preferred where objective evidence is available; model-assisted judgments are used only where necessary.
Validation	Compare automated judgments against human/domain-expert judgments on a representative evaluation subset.

10. Constraints and Assumptions

Table 7 - Constraints and Assumptions

Category	Constraint / Assumption
Technical	Initial implementation focuses on JavaScript/TypeScript projects and the selected testing frameworks.
Execution	Docker execution requires available CPU, memory and storage resources; timeouts and limits are enforced.
AI	AI-generated tests and AI-assisted judgments may be incorrect and require validation.
Chatbot Integration	The target chatbot must expose a testable interface and, where relevant, useful action/tool metadata.
Ground Truth	Project-specific rules and expected behavior must be available or confirmed by a user/domain expert.
Safety	High-risk actions are evaluated in sandbox/mock environments rather than against uncontrolled production systems.
Time	The project is constrained by an academic capstone schedule.
Budget	The project prioritizes open-source tools and low-cost infrastructure.

11. Target Users and Stakeholders

Table 8 - Target Users and Stakeholders

Stakeholder	Needs / Responsibilities
Developer	Analyze source code, test coverage and complexity; generate and validate additional tests.
QA Engineer	Execute and review testing workflows, identify gaps and compare test-quality results.
AI Engineer	Configure chatbot evaluation profiles, adapters, datasets and scoring strategies.
Domain Expert / Product Owner	Confirm business rules, expected answers/actions and high-risk scenarios.
Project Manager / Team Lead	Review project quality indicators, reports, risks and historical comparisons.
Faculty Mentor	Review scope, methodology, progress and academic quality.

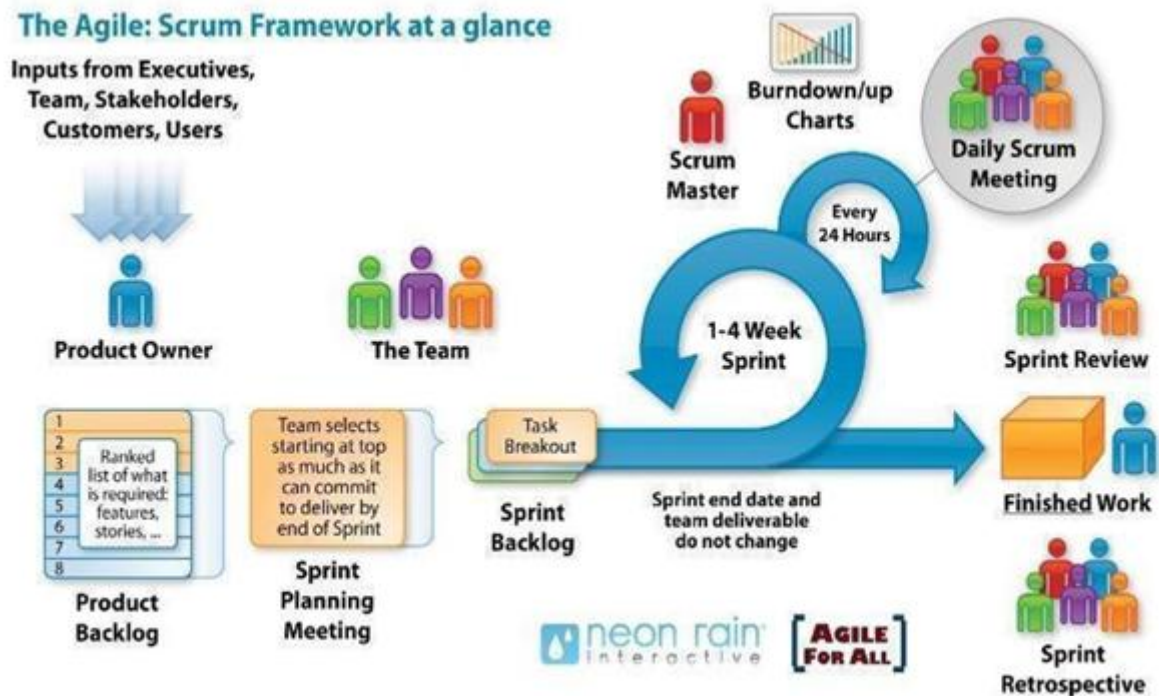
12. Development Methodology and Technology Stack

12.1 Development Methodology (Agile Scrum)

We will follow an Agile Scrum methodology to develop the CovAI platform through iterative and incremental sprints. Each sprint focuses on implementing and integrating a specific group of system features.

Scrum is suitable for this project because the system contains multiple technical components including backend services, frontend interfaces, Docker-based execution, code analysis, AI integration, and testing frameworks.

The iterative development process allows the team to implement core functionality first, evaluate the results, identify technical problems, and gradually expand the system.



Picture 1: Development Methodology (Agile Scrum)

Why did we choose the Scrum process to develop the system?

- We chose Scrum because the project contains multiple complex and interdependent features. Requirements may change during development, especially for AI-generated testing and multi-framework support.
- Scrum allows the team to divide the project into manageable sprints, continuously review progress, and adjust implementation priorities based on technical results and feedback.

12.2 Technology Stack

Table 9 - Technology Stack

Area	Technology
Language	JavaScript, TypeScript
Frontend	React, Vite, Tailwind CSS
Backend	Node.js, Express.js
API	REST API
Database	PostgreSQL
ORM	Prisma
AI	Google Gemini

Area	Technology
Testing	Jest, Vitest, Supertest, Playwright, Cypress
Code Analysis	Babel Parser, Control Flow Graph, Cyclomatic Complexity
Execution	Docker
Queue / Realtime	Redis, Socket.IO
Version Control	Git, GitHub
Development Tools	Visual Studio Code, Postman, Figma

13. Timeline

Table 10 - Project Schedule

No.	Task Name	Duration (Days)	Start Date	End Date
—	Project Timeline	131	Aug 10, 2026	Aug 10, 2026
1	Initial Project Phase	7	Aug 11, 2026	Aug 17, 2026
1.1	Kick-off Meeting	1	Aug 18, 2026	Aug 18, 2026
1.2	Discuss Project Idea	1	Aug 19, 2026	Aug 19, 2026
1.3	Proposal Document	1	Aug 20, 2026	Aug 20, 2026
1.4	Project Plan Document	1	Aug 21, 2026	Aug 21, 2026
1.5	Product Backlog	1	Aug 22, 2026	Aug 22, 2026
1.6	User Story	1	Aug 23, 2026	Aug 23, 2026
1.7	Review Document	1	Aug 24, 2026	Aug 24, 2026
2	Development Phase	56	Aug 25, 2026	Oct 19, 2026
2.1	Sprint 1 – Environment Setup, Authentication & Project Management	14	Aug 25, 2026	Sep 07, 2026

No.	Task Name	Duration (Days)	Start Date	End Date
2.2	Sprint 2 – Source Ingestion, Test Execution & Coverage Analysis	14	Sep 08, 2026	Sep 21, 2026
2.3	Sprint 3 – Code Analysis, Multi-Test Framework Support & AI Test Generation	14	Sep 22, 2026	Oct 05, 2026
2.4	Sprint 4 – GitHub Automation, Monitoring & System Integration	14	Oct 06, 2026	Oct 19, 2026
3	Complete Project Documents	28	Oct 06, 2026	Oct 19, 2026
3.1	Project Retrospective Meeting	2	Oct 06, 2026	Oct 19, 2026
4	Final Release Activities	16	Oct 06, 2026	Oct 19, 2026

14. Project Team

Table 11 - Project Team Roles and Responsibilities

Full Name	Project Role	Primary Responsibilities
Dinh, Huynh Tan	Team Leader / AI & Backend Lead	System architecture, backend core services, AI test generation pipeline, and CI/CD infrastructure.
Phuc, Tran Huu	Backend & Integration Developer	Business logic APIs, database schema design, system integration, and end-to-end

Full Name	Project Role	Primary Responsibilities
		testing.
Quy, Nguyen Duy	Data & Full-Stack Developer	Database query optimization, data analysis services, API integration, and user interface modules.
Thuan, Ngo Huu	Frontend Lead, Full-Stack Developer	Frontend architecture (React dashboard), UI/UX design implementation, state management, and client performance.

15. Risk Management

Table 12 - Risk Assessment

Risk	Type	Probability	Impact	Mitigation
AI-generated tests are incorrect or incomplete	Technical	Medium	High	Validate generated tests, attempt execution, and provide fallback/TODO output.
Test execution fails due to dependencies/configuration	Technical	High	High	Use isolated Docker environments, preflight checks, timeouts and clear logs.
Multi-framework support increases complexity	Technical	Medium	High	Use framework-specific adapters and

Risk	Type	Probability	Impact	Mitigation
				prioritize the selected frameworks.
Large projects consume excessive resources	Performance	Medium	High	Use job queues, time/resource limits and snapshot reuse.
Automated chatbot scoring is biased or inconsistent	AI / Research	Medium	High	Prefer deterministic checks, store evidence, calibrate scoring and compare against human judgments.
Ground-truth business rules are incomplete or outdated	Domain	Medium	High	Require user/domain-expert confirmation and version the evaluation profile.
Tool/action evaluation causes unintended side effects	Safety	Low	Very High	Use mock/sandbox tools, non-production accounts, confirmation gates and forbidden-action rules.
Sensitive evaluation data is exposed	Security	Medium	High	Apply authorization, data minimization and avoid unnecessary external transmission.

Risk	Type	Probability	Impact	Mitigation
Limited academic development time	Management	High	High	Prioritize core features and deliver in Scrum increments.
Target chatbot does not expose sufficient metadata	Integration	Medium	Medium	Use a configurable adapter and clearly mark unavailable evidence/criteria.

16. Budget and Resources

16.1 Budget

Table 13 - Budget and Resource Allocation

Category	Item & Description	Pricing Model / Status	Estimated Cost (VND)
Hardware	Personal development machines (Laptops/PCs)	Existing student-owned equipment	0 VND
Infrastructure	Cloud VPS / Database hosting (Testing & Demo)	Paid cloud instance (monthly/quarterly)	250.000 – 400.000 VND
Domain & Network	Custom domain name (.com / .tech / .site) & SSL	1-year registration / Let's Encrypt	150.000 – 250.000 VND
AI Services	Gemini API & LLM integration	Google AI Studio (Free tier quota) + buffer pay-as-you-go	100.000 VND
Software & Tools	IDEs, Git repository, Docker, Open-source libs	GitHub Education / Open-source licenses	0 VND
Academic Deliverables	Poster printing, capstone thesis binding, demo materials	Academic submission expenses	200.000 – 300.000 VND

Category	Item & Description	Pricing Model / Status	Estimated Cost (VND)
Contingency	Miscellaneous & service overage reserve (~10%)	Contingency buffer	100.000 VND
Total Estimated Budget		800.000 – 1.150.000 VND	

16.2 Human Resources

The project is fully executed by an internal team of four undergraduate software engineering students under faculty supervision. Because this is an academic research and capstone project, labor expenses are treated as non-billable academic effort:

- Project Team: 4 software engineering students.
- Expected Effort: Approximately 15–20 hours/week per member across a 15-week development lifecycle (total estimated effort: ~1,000–1,200 Man-hours).
- Supervision: Academic and technical advising provided by the faculty project supervisor.

17. Conclusion

COVAI is designed as a unified software-quality platform with two major capabilities: automated software testing and AI chatbot trustworthiness evaluation. The software-testing pipeline helps developers move from source-code ingestion to test execution, coverage analysis, structural analysis and AI-assisted test generation. The AI-evaluation pipeline complements this by checking whether an internal chatbot behaves according to project-specific facts, rules, context and action constraints.

The combination of project snapshots, background jobs, evidence collection, configurable evaluation profiles and reporting provides a basis for repeatable quality assessment. At the same time, the project explicitly recognizes the limits of AI-generated tests and model-assisted evaluation, so critical results remain subject to validation and domain-expert review.

The expected result is a functional academic prototype that demonstrates how software testing quality and AI-powered application behavior can be evaluated in one engineering workflow.

18. References

Table 14 - References

No.	Title	Link
1	Scrum Guide	https://en.wikipedia.org/wiki/Scrum_(software_development)
		https://www.atlassian.com/agile/scrum
		https://www.digite.com/agile/scrum-methodology/
		https://www.scrum.org/resources/scrum-guide
2	React Documentation	https://react.dev/reference/react
3	Node.js Documentation	https://nodejs.org/docs/latest/api/
4	Express.js Documentation	https://expressjs.com/
5	Prisma Documentation	https://www.prisma.io/docs/prisma-orm/quickstart/postgresql
6	Jest Documentation	https://jestjs.io/docs/getting-started
7	Vitest Documentation	https://vitest.dev/guide/
8	Playwright Documentation	https://playwright.dev/docs/intro
9	Cypress Documentation	https://docs.cypress.io/app/get-started/why-cypress
10	Docker Documentation	https://docs.docker.com/guides/
11	GitHub Documentation	https://docs.github.com/en/get-started
12	Google Gemini Documentation	https://ai.google.dev/gemini-api/docs

No.	Title	Link
13	ISO/IEC/IEEE 29119 – Software Testing	https://www.iso.org/standard/81291.html
14	ISO/IEC 25010 – Systems and Software Quality Models	https://www.iso.org/standard/78176.html
15	ISO/IEC/IEEE 12207 – Software Life Cycle Processes	https://www.iso.org/standard/81702.html
16	ISTQB Glossary	https://glossary.istqb.org/en_US/search?term=&exact_matches_first=true

END

CAPSTONE PROJECT REGISTRATION FORM*(For CMU-SE 450 / 451 – Software Engineering Program)***1. General Information**

Item	Details
Project Title	CovAI - Software Testing and AI Chatbot Trustworthiness Evaluation System
Project Type	☐ Industry-based ☒ Research-based ☐ Startup/Product ☐ Others: ____
Capstone Course	☒ CMU-SE 450 ☐ CMU-SE 451 ☐ CMU-IS 450 ☐ CMU-IS 451 ☐ CMU-CS 450 ☐ CMU-CS 451
Semester / Year	Semester 1 / 2026 - 2027
Proposed by	☒ Student Team ☐ Faculty ☐ Industry Partner

2. Team Information

No.	Full Name	Student ID	Email	Role (Leader/Dev/ QA/BA/...)
1	Huynh Tan Dinh	29219052500	huynhtandinh.dev@gmail.com	Leader – Dev
2	Tran Huu Phuc	29211458179	phuc12435az@gmail.com	Developer – QA
3	Nguyen Duy Quy	29211158823	quylavip333@gmail.com	Developer – BA
4	Ngo Huu Thuan	29219065178	thuanhuugl@gmail.com	Developer – QA

3. Supervisor / Mentor

Role	Name	Organization	Email
Faculty Mentor	Nguyen Duc Man	International School, Duy Tan University	mannd@duytan.edu.vn

4. Problem Statement

- **Background of the problem:**

Software quality assurance relies heavily on comprehensive testing across multiple application layers (Unit, Integration, and System/E2E). As software applications become increasingly complex, maintaining high test coverage remains a primary defense against bugs and security vulnerabilities. Modern software engineering teams adopt Continuous Integration and Continuous Deployment (CI/CD) practices to deliver features rapidly. However, ensuring that code changes are properly verified across all testing layers without slowing down deployment pipelines remains a critical challenge. At the same time, web applications are increasingly integrating internal AI chatbots for knowledge assistance, user guidance, and tool/API-based actions. The reliability of these chatbots cannot be determined only by the name of the underlying model. Their behavior also depends on system prompts, internal data, retrieval mechanisms, conversation context, tool permissions, refusal logic, and implementation choices. Two chatbots that use the same model may therefore produce different levels of reliability and require project-specific evaluation against verified ground-truth.

- **Current limitations / gaps:**

- **Passive Coverage Analytics:** Traditional coverage tools (e.g., Istanbul, NYC, Jacoco) operate strictly as passive monitors. They generate static numeric reports (uncovered

lines and branches) but offer no actionable remediation, leaving developers to manually decipher and write missing test cases.

- **Granularity Mismatch in AI Test Generators:** Existing AI-assisted test generation tools focus almost exclusively on isolated Unit Tests. They fail to evaluate higher-tier integration levels (such as API endpoints, controller responses, and end-to-end user journeys), leaving critical architectural boundaries untested.
 - **Lack of Closed-Loop Automation:** Most coverage evaluation tools are disconnected from real-time development workflows. They lack execution-validated feedback loops that can automatically detect coverage drops during git push events, generate corrective tests, and verify their execution within isolated environments.
 - **Absence of Project-Specific Chatbot Trustworthiness Verification (AI Reliability Focus):** General-purpose comparisons of large AI models are too broad because different tasks require different evaluation criteria. In contrast, an internal chatbot operates inside a specific web application where source code, business documents, FAQ lists, and API specifications define the ground truth. There is currently a complete lack of automated, reusable testing systems that can verify whether an internal chatbot behaves correctly, faithfully, safely, consistently, and appropriately within the precise data boundaries and business rules of its own hosting application.
- **Target users / stakeholders:**
 - **Software Engineers & Developers:** Developers who require immediate, actionable feedback on untested code paths and automated test suggestions to meet rigorous code quality gates without manual overhead.
 - **QA & Automation Engineers:** QA teams who need a consolidated multi-tier testing overview (Unit, Integration, E2E) and automated assistance in authoring complex API and system integration tests.
 - **AI Engineers & Quality Assessors (AI Reliability Focus):** AI engineers who need automated, repeatable mechanisms to configure chatbot evaluation profiles, ingest business

knowledge bases, generate benchmark scenarios, execute evaluations, and analyze performance across versions.

- **Domain Experts, Product Owners & Engineering Managers (AI Reliability Focus):** Stakeholders who demand automated, objective trust and compliance enforcement for integrated AI features. They require a centralized platform to confirm business rules, expected facts, and forbidden actions, and to monitor chatbot quality scores to prevent legal, financial, and reputational liabilities from unaligned AI behavior.

5. Project Objectives

- Design and implement an automated multi-tier coverage analysis pipeline integrated with GitHub Webhooks that dynamically detects project test ecosystems (Jest, Vitest, Supertest, Playwright, Cypress) and computes real-time runtime coverage metrics (Line/Branch %, API Endpoint %, and Scenario Execution %) within 3 minutes of code push events.
- Develop a targeted AI-driven test generation engine utilizing Abstract Syntax Tree (AST) context extraction and execution-guided self-healing loops to automatically generate missing test cases for uncovered code paths, achieving a minimum 90% execution pass rate without human intervention in sandbox runner environments.
- Deliver an automated CI/CD remediation workflow and dashboard that tracks versioned coverage snapshots across commit SHAs and automatically issues fully validated GitHub Pull Requests containing generated test suites whenever a commit decreases overall repository coverage by more than 5%.
- Design and establish an automated evaluation engine based on a robust three-layer evaluation framework (Core Criteria, Task Modules, and Project Rules). It will ingest project-specific business knowledge assets, generate comprehensive benchmark scenarios, execute automated conversational runs, and compute granular trust scores (correctness, safety, faithfulness, refusal logic) with versioned side-by-side comparison.

6. Proposed Solution

- **System overview**

COVAI is a unified, AI-driven software quality engineering platform that combines two complementary, high-quality capabilities: automated software test execution/analysis and AI chatbot trustworthiness evaluation into a single, integrated development workflow. For software testing, COVAI ingests codebases (ZIP or GitHub), statically analyzes project structure to detect the testing framework, executes tests within isolated Sandbox Docker containers, and analyzes code coverage and structure (CFG, complexity). It then utilizes a self-healing AI loop to generate missing test cases. For AI chatbot evaluation, COVAI ingests business documents, FAQ lists, and API specifications to build a structured evaluation profile. It automatically designs benchmark scenarios across multiple chatbot tasks (Knowledge Q&A, Usage Guidance, and Action/Tool Execution), runs automated conversational test scenarios, and scores the chatbot's responses using a combination of model-assisted (Gemini-as-a-Judge) and deterministic evaluations.

- **Key features:**

- **Automated Ecosystem Detection:** Statically analyzes project manifests (package.json, configuration signatures) to identify existing testing frameworks across Unit (*Jest*, *Vitest*), Integration (*Supertest*, *Playwright API*), and System/E2E (*Cypress*, *Playwright E2E*) layers.
- **Isolated Multi-Tier Execution & Metrics Extraction:** Runs test suites inside lightweight, isolated Sandbox containers to capture true runtime coverage metrics (Line/Branch %, API Endpoint %, and Scenario Execution %).
- **AST-Based Context Extraction:** Parses AST structures to extract precise function signatures, DTOs, and uncovered line boundaries, minimizing token consumption while providing rich context for AI prompts.
- **Execution-Guided Self-Healing AI Loop:** Sends uncovered code contexts to LLMs to generate missing test cases, automatically runs the generated tests in the Sandbox, captures any runtime/syntax errors, and recursively prompts the LLM to fix failed assertions until a 100% execution pass rate is achieved.
- **Automated Pull Request Remediation:** Automatically creates formatted GitHub Pull Requests containing verified test suites when coverage falls below configured quality thresholds.

- **Commit-SHA Snapshot Tracking:** Stores and visualizes historical coverage trends across commit SHAs, enabling side-by-side snapshot diffing on a centralized Web Dashboard.
 - **Multi-Source Knowledge Ingestion & Profile Builder (AI Reliability Focus):** Ingests project-specific knowledge assets (TXT, PDF, FAQ tables, and OpenAPI specs) to construct a structured evaluation profile mapping out application rules, target chatbot capabilities, and expected user behavior.
 - **Automated Benchmark Scenario Generator (AI Reliability Focus):** Automatically generates test prompts and scenarios categorized by chatbot tasks: Knowledge Q&A (retrieval validity), Usage Guidance (correct step-by-step UI sequence), and Action/Tool Execution (API parameter accuracy and tool calling constraints).
 - **Three-Layer AI Evaluation Scoring Engine (AI Reliability Focus):** Implements a rigorous evaluation framework: Layer 1 (Core conversational criteria: correctness, faithfulness, relevance, safety, privacy, refusal), Layer 2 (Task module criteria: tool selection, step sequence), and Layer 3 (Project specific rules: compliance, roles/permissions).
 - **Multi-Configuration Version Comparison & Diagnostics (AI Reliability Focus):** Allows developers and AI engineers to run side-by-side trustworthiness benchmarks across different chatbot versions, configurations, or prompt designs, classifying failed conversational cases and suggesting targeted model improvements.
- **Expected outputs:**
 - **Multi-Tier Coverage Reports:** Consolidated JSON/HTML artifacts and dashboard visualizers summarizing Line/Branch Coverage (Unit), Endpoint/Controller Coverage (Integration), and Scenario Path Coverage (System).
 - **Executable AI-Generated Test Suites:** Fully validated, error-free test files (*.test.js, *.spec.ts) tailored to the project's detected testing framework with valid assertions.
 - **Automated GitHub Pull Requests:** Ready-to-merge PRs containing new test cases targeting previously uncovered code paths triggered by coverage regressions.
 - **Historical Snapshot Ledger:** An indexed database record of coverage metrics tied to repository Commit SHAs, tracking coverage growth and regression trends over time.

- **Interactive AI Chatbot Evaluation Profile & Benchmark Scenarios (AI Reliability Focus):** Version-controlled JSON/YAML benchmark scenario datasets containing specific prompts, context payloads, expected facts, and tool-calling boundaries corresponding to the project's business documents and rules.
- **Chatbot Trustworthiness Reports & Diagnostic Logs (AI Reliability Focus):** A highly visual report demonstrating overall trust scores, component-level criteria scores, classified conversation failure modes, prompt latency/cost logs, and actionable remediation suggestions.

7. Complex Engineering Problem Justification

(MANDATORY – for approval)

The project must satisfy at least three of the following criteria:

Criteria	Yes/No	Explanation
Involving wide-ranging or conflicting technical issues. (e.g., security vs performance)	Yes	Execution Safety vs. Performance & Token Cost: Running unverified AI-generated code inside sandbox environments presents severe security risks and high latency. The system must balance strict container isolation and multi-step self-healing execution loops against tight CI/CD time limits (under 3 minutes) and LLM token budget constraints. Furthermore, evaluating AI chatbots requires running multiple conversational tests in parallel while maintaining strict sandboxing for tool/action execution to avoid real-world side effects without compromising testing latency.
No obvious existing solution	Yes	Existing tools either provide passive coverage numbers without remediation (Codecov, SonarQube) or generate isolated unit tests without runtime validation. There is no unified solution that integrates multi-tier runtime coverage extraction (Unit, API, E2E),

Criteria	Yes/No	Explanation
		static ecosystem auto-detection, closed-loop self-healing AI test generation, and project-specific three-layer AI chatbot evaluation in a single automated workflow.
Involving diverse groups of stakeholders.	Yes	The system must satisfy conflicting requirements across different operational roles: Software Developers demand non-intrusive automation and zero false positives; QA Engineers require accurate multi-layer coverage granularity; DevOps Managers prioritize pipeline stability and cost; AI Engineers require fine-tuning evaluation adapters and automated scoring; and Domain Experts/Product Owners require verified alignment with business rules to protect brand trust.
Multi-technology integration (AI, IoT, Big Data, etc.)	Yes	Integrates Generative AI / LLMs (for both code generation and model-assisted chatbot trustworthiness evaluation), DevOps & Infrastructure (Docker containerization, sandbox isolation, GitHub Webhooks), Static Code Analysis (AST syntax parsing), and Web Full-Stack Technologies (real-time dashboards, BullMQ background

Criteria	Yes/No	Explanation
		queues, and snapshot ledger management).
Significant societal/industrial impact	Yes	Directly reduces software maintenance costs and human error in mission-critical applications. By automating the tedious task of authoring integration/system tests and verifying that customer-facing AI chatbots adhere to safety, privacy, and business policies, it accelerates software delivery while preventing costly production outages and reputational/legal liabilities from unaligned AI behavior.
Addressing problems not encompassed by current standards and codes.	Yes	Industry testing standards define what coverage metrics mean (e.g., ISO/IEC/IEEE 29119) but fail to standardize how LLM-generated code should be validated for semantic correctness and assertion quality within automated CI/CD loops, or how LLM chatbot responses should be objectively scored for context retention, faithfulness, and tool-calling accuracy.
Involving multiple disciplines.	Yes	Combines Software Engineering (test architecture and design patterns), Artificial Intelligence / NLP (context truncation, LLM-as-a-judge scoring,

Criteria	Yes/No	Explanation
		semantic similarity), Cybersecurity (secure sandbox execution, credential scrubbing, PII protection), and Systems Architecture (event-driven microservices, background job queues).

8. System Constraints & Impact Analysis

(*CRITICAL SECTION – heavily weighted*)

8.1 Engineering Constraints

- ☒ Performance
- ☒ Security & Privacy
- ☒ Scalability
- ☒ Cost
- ☒ Maintainability
- ☒ Economic
- ☐ Ethical
- ☐ Public health, safety, and welfare
- ☐ Social and Global
- ☐ Cultural
- ☐ Sustainability
- ☐ Other: please specify.....

Explain how system addresses them:

Performance:

Running multi-tier test suites inside containers can introduce CI/CD pipeline latency. The system mitigates this by using lightweight Docker containers, enforcing strict execution timeouts (under 3 minutes per push), and applying AST parsing to extract only relevant code signatures. For chatbot evaluation, multi-threaded prompt execution is decoupled using BullMQ/Redis background queues to prevent blocking user interfaces.

Security & Privacy:

Executing unverified AI-generated code presents arbitrary code execution risks. All test executions are isolated within network-restricted, ephemeral Docker containers running with non-root privileges. To ensure data privacy, sensitive credentials, API keys, and PII are strictly scrubbed from source code and chatbot conversation histories before prompt payload construction.

Scalability:

High-frequency commit activity and large chatbot evaluation runs (consisting of hundreds of generated benchmark scenarios) can easily overload backend workers. An event-driven task queue (BullMQ/Redis) decouples incoming GitHub Webhooks and evaluation triggers from backend workers, enabling horizontal scaling of runner containers.

Cost:

Frequent Large Language Model (LLM) API calls risk high operational token expenses. The system constrains costs through AST-based context truncation (passing only uncovered code slices rather than whole files) and prompt caching/token minimization during chatbot evaluations.

Maintainability:

To prevent framework lock-in, the system follows a modular microservices architecture. The static parser, container runner, LLM provider, and chatbot evaluation adapter layers are decoupled via abstract interfaces, allowing seamless swapping between AI models (OpenAI, Gemini, Llama) or test runners without refactoring core business logic.

Economic:

Manual test writing and manual chatbot testing account for significant developer/QA labor costs. By automating multi-tier coverage analysis, generating ready-to-merge Pull Requests, and automatically verifying chatbot trustworthiness against business rules, the system lowers QA overhead by up to 30% and accelerates product release cycles.

8.2 Global & Societal Impact

- Cultural / Social impact

Software development and AI teams are increasingly distributed across global time zones. By providing automated, objective quality enforcement and standardized test/chatbot evaluation via automated workflows, COVAI reduces communication friction, eliminates quality disparities between remote teams, and fosters an engineering culture focused on safety and alignment. Furthermore, it builds public trust in AI applications by preventing rogue chatbot behaviors.

- Environmental (Green IT)

To reduce the carbon footprint associated with heavy cloud computing and LLM inference, the system enforces energy-efficient execution strategies. It optimizes compute utilization through local AST pre-processing, prompt caching, and ephemeral container destruction, minimizing redundant energy and data center resource consumption.

- **Economic impact**

Uncaptured software bugs and AI chatbot hallucinations cost the global IT industry billions annually in emergency hotfixes, customer churn, and legal liabilities. By enforcing multi-tier software coverage and rigorous chatbot alignment early in the development lifecycle, the system mitigates technical debt, reduces manual QA costs, and accelerates time-to-market.

8.3 Ethical Considerations

- **Data privacy**

The system strictly enforces a zero-trust data pipeline. Proprietary source code and chatbot transcripts are never stored permanently on third-party AI servers. All environment variables, API tokens, and sensitive data are scrubbed before payload construction. Only anonymized AST structures and cleaned chatbot responses are transmitted to the evaluation API.

- **AI bias / fairness**

AI models may exhibit bias toward specific coding styles or evaluation parameters. COVAI addresses this by employing framework-specific prompt templates, utilizing deterministic validator rules where objective evidence exists, and calibrating LLM-as-a-judge scoring by comparing automated judgments against human/domain-expert judgments on a representative subset.

- **Intellectual property**

All test scripts and evaluation benchmark datasets generated by the system become the exclusive intellectual property of the project owner. The system relies on permissively licensed open-source test runners (Jest, Supertest, Cypress) and ensures that generated test suites do not inject copyrighted external dependencies into the target codebase.

9. Standard requirements

- ☒ Code standards (GNU Coding Standards, clean code, typing, and formatting)
- ☒ Design standards (system architecture, OOAD, and design patterns)
- ☒ IEEE standards (1058, 1540, 830, 1016, 829, 1012, 1008)
- ☒ ISO/IEC/IEEE 12207:2017 (TCVN 10539:2014); ISO/IEC 25051:2006 (TCVN 10540:2014)
- ☐ Other standards related to specific topics.

10. Technical Stack & Architecture

Category	Proposed Technology
Frontend	React.js, TypeScript, Tailwind CSS, Recharts
Backend	Node.js, Express.js, TypeScript, BullMQ (task queue), Babel/Parser (AST parsing)
Database	PostgreSQL, Prisma ORM, Redis (queue & caching)
DevOps / CI-CD	Docker (sandbox runner), GitHub Webhooks, GitHub REST API (Auto PR), Microsoft Azure (VM hosting)
Tools (Jira, Git, etc.)	Git, GitHub, Jira, Postman, Figma, Visual Studio Code.

10. Development Methodology

- ☒ Agile/Scrum
- ☐ V-Model
- ☐ Hybrid

Sprint plan (high-level):

Sprint 1 – Project Setup & Authentication: Set up the development environment, baseline database schema (supporting projects, snapshots, users, and chatbot evaluation profiles), JWT and GitHub OAuth authentication APIs.

Sprint 2 – Project Management & Source Ingestion: Implement Project CRUD, ZIP upload and security scanning, snapshot management, GitHub repository import, and chatbot knowledge asset ingestion (business documents, FAQ lists, and OpenAPI specifications upload and parsing).

Sprint 3 – Unit Test Analysis: Implement Docker-based test runner, Jest/Vitest detection and execution, Unit Test coverage parsing, Control Flow Graph generation, and Cyclomatic Complexity analysis.

Sprint 4 – Integration & System Test Analysis: Implement Integration testing (Supertest, Playwright API) and System testing (Cypress, Playwright E2E) and analyze ingested project

contexts to construct the AI Chatbot Evaluation Profile (relevant business rules, expected behaviors, and task classifications).

Sprint 5 – AI Test Generation: Implement AI-powered test skeleton and full test generation (AST-based, self-healing loop) and automatically generate AI Chatbot evaluation scenarios (Knowledge Q&A, Usage Guidance, and Action/Tool test prompts).

Sprint 6 – GitHub Automation & Requirement Traceability: Implement GitHub Webhook-triggered software analysis and implement the Chatbot Evaluation Runner (connecting to target chatbot APIs, sending prompts, and capturing response transcripts/metadata).

Sprint 7 – Advanced Code Quality & Visualization: Automate GitHub Pull Request creation for coverage drops and implement the three-layer AI scoring engine (Layer 1 core criteria, Layer 2 task modules, Layer 3 project rules) using Gemini-assisted and deterministic checks.

Sprint 8 – Notification, CI/CD & Finalization: Implement system notifications, unified dashboard (software testing metrics + chatbot trust metrics charts), performance optimization, system integration, testing, and final documentation.

11. Major Deliverables Plan

Phase	Deliverable
Proposal	Project Proposal
Design	SRS + SAD
Implementation	Prototype + Test
Final	Report + Demo

12. Risk Assessment

Risk	Impact	Mitigation
Test Execution Failure	Tests may fail due to missing dependencies, configuration errors, or incompatible project environments, resulting in inaccurate analysis.	Run tests in isolated Docker containers, apply timeouts, capture logs, and handle execution failures.
Unsupported or Incorrect Framework Detection	The system may run the wrong testing framework or be unable to analyze the project.	Automatically detect dependencies, configuration files, and test files before selecting the appropriate runner.

Risk	Impact	Mitigation
AI-Generated Test Errors	Medium	Validate generated tests by executing them in Docker and provide self-healing LLM prompt loops or fallbacks when generation fails.
Automated Chatbot Scoring Bias/Inconsistency	High	Prefer deterministic checks, store raw conversation transcripts, calibrate scoring prompts, and validate automated scores against human judgments on a representative subset.
Ground-Truth Rules Incomplete/Outdated	High	Require explicit domain-expert confirmation and version-control the evaluation profiles alongside project snapshots.
Tool/Action Chatbot Evaluation Side-Effects	Very High	Execute actions strictly inside mock/sandbox environments, and enforce forbidden-action filters and human confirmation gates for high-risk scenarios.
Security Risks & Sensitive Data Leakage	High	Apply strict sandboxing, scan uploads, and scrub credentials/PII from prompt payloads before sending to the LLM API.
Project Scope & Schedule Overrun	High	Prioritize core features, implement advanced features incrementally, and deliver in Scrum increments across the Capstone 1 & 2 lifecycle.

13. Expected Contributions

- Technical contribution

- **Unified Web Platform:** Develop a web-based portal for automated software test analysis, coverage measurement, and AI chatbot trustworthiness evaluation.
 - **Multi-Framework Software Testing Support:** Integrate Jest, Vitest, Supertest, Playwright, and Cypress inside Docker-based execution environments.
 - **Modular AI Chatbot Trustworthiness Evaluation Engine:** Build a reusable, three-layer chatbot evaluation framework capable of scoring conversational interfaces against deterministic and model-assisted metrics.
- Innovation / novelty
 - **Ecosystem Integration:** A single platform combining traditional software quality assurance metrics (coverage, complexity, CFG) and conversational AI reliability (trustworthiness, safety, policy alignment) into a single unified engineering workflow. Use AI to identify coverage gaps and generate appropriate test cases when test files are missing.
 - **Dynamic Evaluation Profile Builder:** Automatically extracts context from business documents, FAQs, and API contracts to compile custom, project-specific conversational benchmark scenarios.
 - **Closed-Loop AST Remediation:** A self-healing test generation loop that uses compilation errors and assertions to continuously refine generated test files.
 - Practical applicability
 - Help developers quickly identify insufficiently tested code paths, improving traditional test suites automatically.
 - Prevents production failures, policy breaches, hallucinations, and unaligned responses in integrated AI chatbots.
 - Enables QA managers and product owners to track traditional software health metrics alongside AI safety indices under a single, unified visual dashboard.

14. Approval Section (For Department Use)

Role	Name	Decision	Signature	Date
Faculty Mentor		☐ Approved ☐ Revise ☐ Reject		
Program Chair		☐ Approved ☐ Revise ☐ Reject		

CAPSTONE PROJECT PROPOSAL EVALUATION RUBRIC
(SE Program – Approval Stage)

Project title:

TeamID:

Date:

◆ **Evaluation Criteria**

No.	Criteria	Weight	Key Focus	Score/mark
1	Problem Definition	15%	Clear, real-world relevance, stakeholders	
2	Complex Engineering Problem	20%	Meets ≥3 complexity characteristics	
3	Solution Feasibility	15%	Technical soundness, initial architecture	
4	Design Constraints	15%	Security, performance, cost, scalability ...	
5	Global & Ethical Impact	10%	Ethics, societal, environmental factors	
6	Innovation	10%	Novelty, originality	
7	Scope & Planning	10%	Feasibility within 14–16 weeks	
8	Team Structure	5%	Roles, capability balance	
9	Total score	100%		

Scale: 1 (Poor) – 5 (Excellent)

◆ **Final Decision**

Total Score	Decision
≥ 75	✓ Approved
60–74	⚠ Revise & Resubmit
< 60	✗ Reject

◆ **Comments**

- Strengths:
- Weaknesses:
- Required Revisions:

Evaluator's Signature

AI USAGE DISCLOSURE STATEMENT

(Software Engineering/MIS/Network Security Capstone Project)

University: Duy Tan University

Faculty / Program: Software Engineering CMU

Course: Capstone Project / Graduation Project

Project Title: COVAI – Software Testing and AI Chatbot Trustworthiness Evaluation System

Team Name: C1SE.30

Mentor: Man, Nguyen Duc, Ph.D.

Student ID	Full Name	Role in Project
29219052500	Dinh, Huynh Tan	Team Leader / AI & Backend Lead
29211458179	Phuc, Tran Huu	Backend & Integration Developer
29211158823	Quy, Nguyen Duy	Data & Full-Stack Developer
29219065178	Thuan, Huu Ngo	Frontend Lead, Full-Stack Developer

1. AI TOOLS DECLARATION

Please list all AI tools that may be used in this project.

AI Tool	Version / Platform	Intended Purpose
ChatGPT	GPT-4o / Web & Desktop	Outlining sections, refining academic English syntax, drafting technical descriptions, rephrasing SRS/NFR requirements, Agile/Scrum workflow assistance, and formatting tables.
Google Gemini	Gemini 8 Flash (Web)	Literature review summarization, comparing related works, academic tone refinement, proofreading, and vocabulary enhancement.
GitHub Copilot	VS Code Extension	Intelligent code auto-completion, scaffolding starter templates, drafting API boilerplate, and exploring implementation patterns.
Claude	Claude 3.5 Sonnet / Web	Assisting in debugging syntax errors, IEEE/ISO requirement formatting, architectural component relationship drafting, and entity attribute polishing.
Antigravity	Antigravity IDE / Agentic AI	Intelligent code auto-completion, scaffolding starter templates, drafting API boilerplate, debugging syntax errors, and exploring implementation patterns.

AI Tool	Version / Platform	Intended Purpose
Gemini API	API (Runtime System Component)	Integrated directly into the backend system as an automated service component for test case generation, text summarization, and coverage analysis.
CodeMaid & AI Diagramming Extensions	VS Code / IDE Extensions	Analyzing source structures, reorganizing class layouts, and assisting in visual generation/formatting of architectural diagrams, code flow, and class structures.

Examples: ChatGPT, Claude, GitHub Copilot, Gemini, Midjourney, Cursor AI, etc.

2. EXPECTED AI USAGE BY PROJECT PHASE

Indicate how AI tools may assist during development.

Development Phase	AI Tool (if any)	Intended Use
Problem Analysis	ChatGPT, Google Gemini	Brainstorming sentence structures, academic vocabulary refinement, and articulating technical pain points clearly in English.
System Design	Claude, ChatGPT, AI Diagramming Extensions	Drafting component relationship descriptions, refining technical explanations of system layers, polishing entity attribute explanations, and visual diagram formatting aid.
Implementation	GitHub Copilot, Claude, Antigravity	Intelligent code auto-completion, scaffolding starter templates, drafting API boilerplate, and debugging syntax errors. Core architecture, business logic, security controls, and database transactions are designed, written, and manually reviewed by team members.
Debugging	ChatGPT, Claude, GitHub Copilot	Analyzing error stack traces, diagnosing syntax exceptions, and exploring troubleshooting patterns during local development.
Testing	Gemini API (Runtime Component), GitHub Copilot	Integrated directly into the backend system as an automated service component for test case generation, text summarization, and coverage analysis. Prompts, parsers, and guardrails are engineered and tested by the team.
Documentation	ChatGPT, Google Gemini, Claude	Formatting requirement statements to follow standard IEEE/ISO conventions, categorizing NFR attributes, comprehensive spell-checking, and

Development Phase	AI Tool (if any)	Intended Use
		ensuring stylistic consistency across all chapters.

3. ESTIMATED EXTENT OF AI ASSISTANCE

Provide an estimated percentage of AI-assisted work.

Component	Estimated AI Assistance
Backend / Core Logic	20%
Frontend / UI	20%
Testing	25%
Documentation	30%

Note: Students remain fully responsible for understanding and validating all AI-assisted outputs.

4. RESPONSIBLE AI USE CHECKLIST

Please confirm the following:

Statement	Yes
We will disclose all AI tools used in the project	☒
AI-generated code will be reviewed and understood by the team	☒
AI will be used only as a support tool, not as a full solution generator	☒
All team members can explain the system design and implementation	☒
We agree to demonstrate code ownership during the final defense	☒

5. ACADEMIC INTEGRITY DECLARATION

We acknowledge that:

- AI tools are assistive technologies, not substitutes for engineering work.
- All AI-generated outputs will be reviewed, modified, and validated by the team.
- We take full responsibility for the architecture, implementation, and results of this project.
- Failure to disclose AI usage may be considered academic misconduct.

6. TEAM DECLARATION

We confirm that the information provided above is accurate and that we will follow the Responsible AI Use Policy for Capstone Projects.

Name	Signature	Date
Dinh, Huynh Tan	Dinh	5/9/2026
Phuc, Tran Huu	Phuc	5/9/2026

Quy, Nguyen Duy	Quy	5/9/2026
Thuan, Huu Ngo	Thuan	5/9/2026

AI POLICY FOR SOFTWARE ENGINEERING CAPSTONE PROJECTS

(Responsible and Ethical Use of Artificial Intelligence in Capstone Development)

Institution: Duy Tan University

Faculty / School: International School

Program: Software Engineering / Management Information System / Network Security

Effective Date: Mar 15th, 2026

1. Purpose of the Policy

This policy establishes guidelines for the responsible, transparent, and ethical use of Artificial Intelligence (AI) tools in Software Engineering Capstone projects.

The objectives are to:

1. Encourage productive and responsible use of AI tools in software development.
2. Ensure academic integrity and originality of student work.
3. Prevent misuse of AI that results in superficial or unverified project outcomes.
4. Ensure that students demonstrate genuine technical competence and understanding.
5. Maintain the academic and professional standards expected from a university-level engineering capstone project.

AI tools are recognized as legitimate development aids; however, students remain fully responsible for the intellectual and technical content of their projects.

2. Scope

This policy applies to:

- All **Capstone / Senior Projects / Graduation Projects**
- All students enrolled in:
 - Software Engineering
 - Management Information Systems
 - Related computing programs
- All phases of the project lifecycle:
 - Proposal
 - Design
 - Implementation
 - Testing
 - Documentation
 - Final defense

3. Definition of AI Tools

For the purposes of this policy, AI tools include but are not limited to:

3.1 Generative AI

Examples:

- ChatGPT
- Claude
- Gemini
- LLaMA-based assistants

Uses may include:

- concept clarification
- documentation drafting
- code explanation
- debugging assistance

3.2 AI Coding Assistants

Examples:

- GitHub Copilot
- Amazon CodeWhisperer
- Tabnine
- Cursor AI

Uses may include:

- code suggestion
- autocomplete
- test generation

3.3 AI Design and Media Tools

Examples:

- Midjourney
- DALL·E
- Stable Diffusion
- Figma AI plugins

Uses may include:

- UI/UX concept generation
- visual prototyping

3.4 AI Data Analysis Tools

Examples:

- Python AI libraries
- AutoML tools
- LLM-based data analysis assistants

4. Acceptable Uses of AI

Students are allowed to use AI tools for supportive purposes, including:

4.1 Learning and Understanding

- explanation of programming concepts
- clarification of algorithms
- understanding frameworks and libraries

4.2 Development Assistance

AI may assist with:

- code suggestions
- debugging hints
- refactoring ideas
- generating test cases
- generating boilerplate code

However, students must:

- review
- understand
- modify

all AI-generated content.

4.3 Documentation Support

AI may assist in:

- drafting documentation
- formatting reports
- summarizing references

Students must edit and verify accuracy.

4.4 Brainstorming and Design Ideation

AI can be used to:

- explore architecture patterns
- compare technical solutions

- generate early design ideas

Final design decisions must be made and justified by the student team.

5. Unacceptable Uses of AI

The following uses are strictly prohibited:

5.1 AI-Generated Projects

Submitting a project where:

- the majority of code is generated by AI
- students cannot explain the system

5.2 Blind Code Copying

Using AI-generated code:

- without understanding
- without validation
- without modification

5.3 AI-Generated Documentation Without Verification

Submitting documentation that is:

- entirely AI-generated
- not reviewed by students

5.4 AI-Based Academic Misconduct

Examples include:

- generating full reports using AI
- generating design documents without team involvement
- fabricating research references
- submitting AI-generated work as original intellectual contribution

6. AI Usage Disclosure Requirement

All teams must disclose AI usage in their project documentation.

Disclosure must include:

1. AI tools used
2. purpose of usage
3. project phase where AI was applied

4. extent of AI-generated content

This disclosure must appear in:

- Proposal document
- Final report
- Appendix (AI Usage Statement)

Failure to disclose AI usage may be considered academic misconduct.

7. Student Responsibility

Students remain **fully responsible** for:

- system architecture
- algorithmic design
- code correctness
- testing results
- documentation accuracy

During project defense, students must be able to:

- explain their system architecture
- explain implementation decisions
- debug their system
- modify code when requested

8. Evaluation Criteria Related to AI Usage

Project evaluation will consider:

Criterion	Description
Technical Understanding	Students understand their own system
Original Engineering Work	Evidence of human design and reasoning
Responsible AI Use	AI used as tool, not replacement
Code Ownership	Students can explain and modify code
Documentation Integrity	Report reflects genuine work

If a team cannot explain their code, the project may receive significant penalties.

9. Verification Methods

Supervisors and evaluation committees may verify authenticity by:

- requesting live code explanation
- asking students to modify code during defense

- requesting design justification
- reviewing AI usage logs

Failure to demonstrate ownership may result in:

- grade reduction
- project rejection
- academic integrity investigation

10. Ethical Use of AI

Students must adhere to ethical standards including:

- transparency
- accountability
- intellectual honesty
- respect for intellectual property

AI must be used to enhance learning, not to replace engineering competence.

11. Policy Violations

Violations of this policy may result in:

- grade penalties
- project rejection
- academic integrity proceedings
- disciplinary action according to university regulations

12. Student Acknowledgement

All students must sign the following statement:

"We acknowledge that we have read and understood the AI Policy for Software Engineering Capstone Projects. We commit to using AI tools responsibly, transparently, and ethically, and we accept full responsibility for the technical and intellectual content of our project."

Student Name	Student ID	Signature	Date
Dinh, Huynh Tan	29219052500	Dinh	September 05, 2026
Phuc, Tran Huu	29211458179	Phuc	September 05, 2026
Quy, Nguyen Duy	29211158823	Quy	September 05, 2026
Thuan, Huu Ngo	29219065178	Thuan	September 05, 2026

13. Supervisor Confirmation

Signature: Date: